



INFORME DE TAREA PRÁCTICA

I. PORTADA

Tema:	Consultas SQL con JOIN, filtros y agregaciones
Unidad de Organización Curricular:	PROFESIONAL
Nivel y Paralelo:	Cuarto “B”
Alumnos participantes:	Sandro Iker Fernández Lima Brittany Carla Paredes Chávez Gabriel Alexander Rojas Silva Rodolfo Enrique Zambrano Alban
Asignatura:	Base de datos
Docente:	Ing. José Caiza, Mg.

II. INFORME DE GUÍA PRÁCTICA

2.1 Objetivos

General:

Aplicar consultas SQL utilizando SELECT, WHERE, ORDER BY, INNER JOIN, GROUP BY HAVING, SUM, COUNT, AVG y subconsultas Oracle SQL Developer.

Específicos:

- Desarrollar consultas SQL básicas y avanzadas utilizando instrucciones como SELECT, WHERE y ORDER BY para la recuperación y organización de información en la base de datos VETERINARIA_DB
- Implementar relaciones entre tablas mediante el uso de INNER JOIN, permitiendo integrar información de mascotas, propietarios, veterinarios y consultas dentro del sistema
- Aplicar funciones de agregación y subconsultas utilizando GROUP BY, HAVING, SUM, COUNT y AVG para analizar y obtener resultados estadísticos de la información almacenada en la base de datos

2.2 Modalidad

Presencial

2.3 Tiempo de duración

Presenciales: 3

No presenciales: 0

2.4 Instrucciones

1. Descargar Oracle SQL Developer desde el sitio oficial de Oracle.
2. Instalar Java JDK si el sistema lo solicita.
3. Ejecutar sqldeveloper.exe.
4. Crear una nueva conexión:
 - Usuario: VETERINARIA_DB
 - Contraseña: VETERINARIA_DB
 - Host: localhost
 - Puerto: 1521
 - SID: xe
5. Ejecutar el script SQL de creación de tablas e inserts.

2.5 Listado de equipos, materiales y recursos

Listado de equipos y materiales generales empleados en la guía práctica:

- Oracle SQL Developer



TAC (Tecnologías para el Aprendizaje y Conocimiento) empleados en la guía práctica:

- ☐ Plataformas educativas
- ☐ Simuladores y laboratorios virtuales
- ☐ Aplicaciones educativas
- ☐ Recursos audiovisuales
- ☐ Gamificación
- ☒ Inteligencia Artificial
- Otros (Especifique): _____

2.6 Actividades por desarrollar

Instalación y configuración de Oracle SQL Developer

1. Descargar Oracle SQL Developer desde el sitio oficial de Oracle.
2. Instalar Java JDK si el sistema lo solicita.
3. Ejecutar sqldeveloper.exe.
4. Crear una nueva conexión:
 - Usuario: VETERINARIA_DB
 - Contraseña: VETERINARIA_DB
 - Host: localhost
 - Puerto: 1521
 - SID: xe
5. Ejecutar el script SQL de creación de tablas e inserts.

Nueva / Seleccionar Conexión a Base de Datos

Nombre de Cone... Detalles de Cone...
VETERINARIA_DB system@//localh...

Name: Color

Tipo de Base de Datos:

Información de usuario Usuario de Proxy

Tipo de autenticación:

Usuario: Rol:

Contraseña: ☒ Guardar Contraseña

Tipo de Conexión:

Detalles Avanzado

Nombre del Host:

Puerto:

☒ SID:

☐ Nombre del Servicio:

Estado:

Creación de la base de datos: VETERINARIA_BD



The screenshot shows the SQL Developer interface with the 'VETERINARIA_DB' connection selected. The 'Hoja de Trabajo' (Worksheet) tab is active, displaying the following SQL commands:

```
CREATE USER VETERINARIA_DB IDENTIFIED BY VETERINARIA_DB;  
GRANT CONNECT, RESOURCE, UNLIMITED TABLESPACE TO VETERINARIA_DB;
```

Below the code editor, the output window shows the results of the execution:

```
User VETERINARIA_DB creado.  
  
Grant correcto.
```

CREACION DE TABLAS

Creación Tabla Propietario

The screenshot shows the SQL Developer interface with the 'VETERINARIA_DB' connection selected. The 'Hoja de Trabajo' (Worksheet) tab is active, displaying the following SQL command:

```
CREATE TABLE PROPIETARIO (  
    id_propietario NUMBER PRIMARY KEY,  
    cedula VARCHAR2(10) NOT NULL UNIQUE,  
    nombres VARCHAR2(80) NOT NULL,  
    telefono VARCHAR2(15),  
    ciudad VARCHAR2(50) NOT NULL,  
    correo VARCHAR2(100) UNIQUE  
);
```

Below the code editor, the output window shows the results of the execution:

```
Table PROPIETARIO creado.
```

Creación Tabla Especie

The screenshot shows the SQL Developer interface with the 'VETERINARIA_DB' connection selected. The 'Hoja de Trabajo' (Worksheet) tab is active, displaying the following SQL command:

```
CREATE TABLE ESPECIE (  
    id_especie NUMBER PRIMARY KEY,  
    nombre_especie VARCHAR2(50) NOT NULL UNIQUE  
);
```

Below the code editor, the output window shows the results of the execution:

```
Table PROPIETARIO creado.  
  
Table ESPECIE creado.
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: FEBRERO – JULIO 2026



Creación Tabla Especialidad

The screenshot shows the SQL Developer interface with the 'Hoja de Trabajo' (Worksheet) tab active. The SQL script in the editor is:

```
CREATE TABLE ESPECIALIDAD (  
    id_especialidad NUMBER PRIMARY KEY,  
    nombre_especialidad VARCHAR2(80) NOT NULL UNIQUE  
);
```

Below the editor, the 'Salida de Script' (Script Output) window shows the execution results:

```
Tarea terminada en 0,033 segundos  
  
Table PROPIETARIO creado.  
  
Table ESPECIE creado.  
  
Table ESPECIALIDAD creado.
```

Creación Tabla Veterinario

The screenshot shows the SQL Developer interface with the 'Hoja de Trabajo' (Worksheet) tab active. The SQL script in the editor is:

```
CREATE TABLE VETERINARIO (  
    id_veterinario NUMBER PRIMARY KEY,  
    nombres VARCHAR2(80) NOT NULL,  
    cedula VARCHAR2(10) NOT NULL UNIQUE,  
    sueldo NUMBER(8,2) NOT NULL,  
    id_especialidad NUMBER,  
    CONSTRAINT fk_veterinario_especialidad  
        FOREIGN KEY (id_especialidad)  
        REFERENCES ESPECIALIDAD(id_especialidad)  
);
```

Below the editor, the 'Salida de Script' (Script Output) window shows the execution results:

```
Tarea terminada en 0,026 segundos  
  
Table PROPIETARIO creado.  
  
Table ESPECIE creado.  
  
Table ESPECIALIDAD creado.  
  
Table VETERINARIO creado.
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: FEBRERO – JULIO 2026



Creación Tabla Mascota

The screenshot shows the SQL Developer interface with the 'GENERADOR DE CONSULTAS' (Query Generator) window open. The SQL script defines the 'MASCOTA' table with the following columns and constraints:

```
id_mascota NUMBER PRIMARY KEY,  
nombre_mascota VARCHAR2(60) NOT NULL,  
raza VARCHAR2(60),  
edad NUMBER(3) NOT NULL,  
peso NUMBER(5,2) NOT NULL,  
id_propietario NUMBER NOT NULL,  
id_especie NUMBER NOT NULL,  
CONSTRAINT fk_mascota_propietario  
FOREIGN KEY (id_propietario)  
REFERENCES PROPIETARIO(id_propietario),  
CONSTRAINT fk_mascota_especie  
FOREIGN KEY (id_especie)  
REFERENCES ESPECIE(id_especie)
```

Below the script, the 'Salida de Script' (Script Output) window shows the execution results:

```
Table ESPECIE creado.  
  
Table ESPECIALIDAD creado.  
  
Table VETERINARIO creado.  
  
Table MASCOTA creado.
```

Creación Tabla Consulta

The screenshot shows the SQL Developer interface with the 'GENERADOR DE CONSULTAS' (Query Generator) window open. The SQL script defines the 'CONSULTA' table with the following columns and constraints:

```
fecha_consulta DATE NOT NULL,  
diagnostico VARCHAR2(150) NOT NULL,  
tratamiento VARCHAR2(150),  
costo NUMBER(8,2) NOT NULL,  
estado VARCHAR2(30) NOT NULL,  
id_mascota NUMBER NOT NULL,  
id_veterinario NUMBER NOT NULL,  
CONSTRAINT fk_consulta_mascota  
FOREIGN KEY (id_mascota)  
REFERENCES MASCOTA(id_mascota),  
CONSTRAINT fk_consulta_veterinario  
FOREIGN KEY (id_veterinario)  
REFERENCES VETERINARIO(id_veterinario)
```

Below the script, the 'Salida de Script' (Script Output) window shows the execution results:

```
Table ESPECIALIDAD creado.  
  
Table VETERINARIO creado.  
  
Table MASCOTA creado.  
  
Table CONSULTA creado.
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: FEBRERO – JULIO 2026



INSERT INTO

INSERT INTO Propietario

```
INSERT INTO PROPIETARIO
VALUES (1, '1801111111', 'Carlos Pérez', '0991111111')

INSERT INTO PROPIETARIO
VALUES (2, '1802222222', 'Maria López', '0992222222',
```

Salida de Script x | Tarea terminada en 0,029 segundos

1 fila insertadas.

INSERT INTO Especie

```
INSERT INTO ESPECIE
VALUES (1, 'Perro');

INSERT INTO ESPECIE
VALUES (2, 'Gato');
```

Salida de Script x | Tarea terminada en 0,034 segundos

1 fila insertadas.

1 fila insertadas.



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: FEBRERO – JULIO 2026



INSERT INTO Especialidad

The screenshot shows a database management tool window titled 'VETERINARIA_DB'. The 'Hoja de Trabajo' (Worksheet) tab is active, displaying two SQL queries in the 'Generador de Consultas' (Query Generator) area:

```
INSERT INTO ESPECIALIDAD  
VALUES (1, 'Medicina General');  
  
INSERT INTO ESPECIALIDAD  
VALUES (2, 'Cirugia');
```

The 'Salida de Script' (Script Output) window below shows the execution results:

```
1 fila insertadas.  
  
1 fila insertadas.  
  
1 fila insertadas.
```

INSERT INTO Veterinario

The screenshot shows a database management tool window titled 'VETERINARIA_DB'. The 'Hoja de Trabajo' (Worksheet) tab is active, displaying two SQL queries in the 'Generador de Consultas' (Query Generator) area:

```
INSERT INTO VETERINARIO  
VALUES (1, 'Dr. Andrés Ramos', '1701111111', 1200, 1);  
  
INSERT INTO VETERINARIO  
VALUES (2, 'Dra. Paola Vega', '1702222222', 1500, 2);
```

The 'Salida de Script' (Script Output) window below shows the execution results:

```
1 fila insertadas.  
  
1 fila insertadas.  
  
1 fila insertadas.  
  
1 fila insertadas.
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: FEBRERO – JULIO 2026



INSERT INTO Mascota

The screenshot shows the SQL Developer interface with the 'VETERINARIA_DB' database selected. The 'Hoja de Trabajo' (Worksheet) tab is active, displaying the following SQL script:

```
INSERT INTO MASCOTA
VALUES (1, 'Max', 'Labrador', 5, 28.5, 1, 1);

INSERT INTO MASCOTA
VALUES (2, 'Luna', 'Siamés', 3, 5.2, 2, 2);
```

The 'Salida de Script' (Script Output) window at the bottom shows the execution results:

```
1 fila insertadas.

1 fila insertadas.

1 fila insertadas.

1 fila insertadas.
```

The status bar indicates 'Tarea terminada en 0,037 segundos'.

INSERT INTO Consulta

The screenshot shows the SQL Developer interface with the 'VETERINARIA_DB' database selected. The 'Hoja de Trabajo' (Worksheet) tab is active, displaying the following SQL script:

```
1
);

INSERT INTO CONSULTA
VALUES (
2,
TO_DATE('2026-05-03', 'YYYY-MM-DD'),
'Vacunación',
'Vacuna triple felina',
35,
'Finalizada',
2,
2
);
```

The 'Salida de Script' (Script Output) window at the bottom shows the execution results:

```
1 fila insertadas.

1 fila insertadas.

1 fila insertadas.

1 fila insertadas.
```

The status bar indicates 'Tarea terminada en 0,028 segundos'.

Se creo 20 INSERT INTO, 5 creados por cada estudiante



CONSULTAS

Consultas principales

1. Mostrar las mascotas con edad mayor a 4 años.

The screenshot shows a SQL query editor window titled "VETERINARIA_DB". The query is as follows:

```
SELECT *  
FROM MASCOTA  
WHERE edad > 4;
```

Below the query editor, the "Resultado de la Consulta" (Query Result) window is displayed, showing 8 rows of data. The status bar indicates "Todas las Filas Recuperadas: 8 en 0,006 segundos".

ID_MASCOTA	NOMBRE_MASCOTA	RAZA	EDAD	PESO	ID_PROPIETARIO	ID_ESPECIE
1	Max	Labrador	5	28,5	1	1
2	Rocky	Bulldog	6	30,1	3	1
3	Thor	Pastor Alemán	7	35	7	1
4	Toby	Golden Retriever	8	32,4	11	1
5	Lolo	Loro Amazónico	6	1,2	13	4
6	Spike	Iguana Verde	5	4,5	16	6
7	Canelo	Criollo	9	27	17	1
8	Pecas	Tortuga Marina	12	10	19	6

Explicación

Sirve para filtrar y mostrar toda la información de aquellas mascotas que tienen más de 4 años de edad. En términos sencillos, el código utiliza `SELECT *` para indicar que queremos ver todos los datos disponibles de los animales (como su nombre, raza o identificación), `FROM MASCOTA` para especificar que busque dentro de la lista o tabla de mascotas, y la condición `WHERE edad > 4` para que funcione como un filtro que descarta automáticamente a los animales más jóvenes, permitiendo visualizar únicamente a los pacientes que superen estrictamente esa edad.



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: FEBRERO – JULIO 2026



2. Mostrar las consultas atendidas ordenadas por costo.

Hoja de Trabajo							
Generador de Consultas							
<pre>SELECT * FROM CONSULTA ORDER BY costo;</pre>							
Resultado de la Consulta							
SQL Todas las Filas Recuperadas: 19 en 0,012 segundos							
ID_CONSULTA	FECHA_CONSULTA	DIAGNOSTICO	TRATAMIENTO	COSTO	ESTADO	ID_MASCOTA	ID_VETERINARIO
1	2 03/05/26	Vacunación	Vacuna triple felina	35	Finalizada	2	2
2	7 09/05/26	Chequeo general	Vitaminas	40	Finalizada	7	1
3	20 22/05/26	Chequeo preventivo	Vitaminas y control	42	Finalizada	20	3
4	5 07/05/26	Desnutrición	Dieta especial	45	Finalizada	5	1
5	17 19/05/26	Fiebre	Antiinflamatorios	48	Finalizada	17	1
6	1 01/05/26	Infección estomacal	Antibióticos	50	Finalizada	1	1
7	10 12/05/26	Caída de pelo	Shampoo medicinal	55	Finalizada	10	3
8	18 20/05/26	Control de peso	Dieta balanceada	58	Finalizada	18	4
9	13 15/05/26	Parásitos intestinales	Desparasitación	60	Finalizada	13	5
10	9 11/05/26	Infección respiratoria	Antibióticos	65	Finalizada	9	2
11	14 16/05/26	Control rutinario	Chequeo completo	70	Pendiente	14	6
12	3 05/05/26	Alergia en la piel	Crema dermatológica	75	Finalizada	3	3
13	11 13/05/26	Vacunación anual	Vacuna múltiple	80	Finalizada	11	1
14	15 17/05/26	Herida superficial	Cicatrizante	85	Finalizada	15	7
15	4 06/05/26	Problema ocular	Gotas oftálmicas	90	Pendiente	4	7
16	12 14/05/26	Problema dental	Limpieza dental	95	Finalizada	12	2
17	6 08/05/26	Fractura leve	Inmovilización	120	Finalizada	6	6
18	19 21/05/26	Problema respiratorio	Nebulización	130	Pendiente	19	2
19	16 18/05/26	Problema neurológico	Exámenes clínicos	250	Pendiente	16	5

Explicación

Sirve para mostrar un listado completo de todas las atenciones médicas realizadas en la veterinaria, organizándolas de menor a mayor precio. Para lograrlo, el código utiliza `SELECT *` para traer todos los detalles de cada cita y `FROM CONSULTA` para extraer los datos de esa tabla en específico, complementándolo al final con la instrucción `ORDER BY costo`, la cual ordena automáticamente el reporte de forma ascendente para que las consultas más económicas aparezcan primero y las más caras al final.



3. Mostrar mascota, propietario y ciudad.

The screenshot shows a SQL query in a text editor window titled 'VETERINARIA_DB'. The query is as follows:

```
SELECT M.nombre_mascota AS Mascota, P.nombres AS Propietario, P.ciudad AS Ciudad
FROM MASCOTA M
INNER JOIN PROPIETARIO P ON M.id_propietario = P.id_propietario;
```

Below the query, the 'Resultado de la Consulta' (Query Result) is displayed, showing 19 rows of data. The columns are labeled MASCOTA, PROPIETARIO, and CIUDAD.

	MASCOTA	PROPIETARIO	CIUDAD
1	Luna	María López	Quito
2	Chispa	Melissa Guerrero	Riobamba
3	Max	Carlos Pérez	Ambato
4	Milo	Ana Martínez	Riobamba
5	Rocky	Juan Torres	Ambato
6	Coco	Luis Herrera	Cuenca
7	Kiwi	Sofía Andrade	Latacunga
8	Thor	Miguel Chávez	Quito
9	Simba	Pedro Salazar	Guayaquil
10	Pelusa	Valeria Ruiz	Manta
11	Toby	Ricardo Núñez	Ambato
12	Mia	Camila Pérez	Quito
13	Lolo	Fernando Mora	Riobamba
14	Rex	Paula Sánchez	Cuenca
15	Bunny	José Castillo	Latacunga
16	Spike	Andrea Vega	Guayaquil
17	Canelo	Kevin Romero	Manta
18	Mishi	Natalia Ortiz	Ambato
19	Pecas	Esteban Silva	Quito

Explicación

Sirve para cruzar la información de dos listas distintas y mostrar el nombre de cada mascota junto al de su dueño y la ciudad donde viven. Como los nombres de los dueños (nombres) y su residencia están guardados en la tabla PROPIETARIO, y el nombre del animal (nombre_mascota) está en MASCOTA, el código utiliza la instrucción INNER JOIN para conectar ambas tablas a través de su columna común (id_propietario), permitiendo así juntar y presentar los datos relacionados de forma clara en un solo reporte sin que se mezclen.



4. Mostrar consulta, mascota, veterinario y especialidad.

The screenshot shows the SQL Developer interface with a query window titled 'VETERINARIA_DB'. The query is as follows:

```
SELECT C.id_consulta AS Consulta, C.fecha_consulta AS Fecha,  
       M.nombre_mascota AS Mascota,  
       V.nombres AS Veterinario,  
       E.nombre_especialidad AS Especialidad  
FROM CONSULTA C  
INNER JOIN MASCOTA M ON C.id_mascota = M.id_mascota  
INNER JOIN VETERINARIO V ON C.id_consulta = V.id_veterinario  
INNER JOIN ESPECIALIDAD E ON V.id_veterinario = E.id_especialidad;
```

Below the query window, the 'Resultado de la Consulta' window shows the results of the query. It indicates that 8 rows were recovered in 0.026 seconds. The results are displayed in a table with the following columns: CONSULTA, FECHA, MASCOTA, VETERINARIO, and ESPECIALIDAD.

	CONSULTA	FECHA	MASCOTA	VETERINARIO	ESPECIALIDAD
1	2	03/05/26	Luna	Dra. Paola Vega	Cirugía
2	1	01/05/26	Max	Dr. Andrés Ramos	Medicina General
3	3	05/05/26	Rocky	Dr. Luis Herrera	Dermatología
4	4	06/05/26	Milo	Dra. Camila Torres	Cardiología
5	5	07/05/26	Coco	Dr. José Medina	Neurología
6	6	08/05/26	Kiwi	Dra. Ana Flores	Traumatología
7	7	09/05/26	Thor	Dr. Miguel Castro	Oftalmología

Explicación

Sirve para generar un reporte completo de las citas uniendo la información de cuatro tablas distintas en una sola vista. El código utiliza múltiples instrucciones INNER JOIN para encadenar los datos: conecta la CONSULTA con la MASCOTA que fue atendida, luego vincula al VETERINARIO encargado de la cita y, finalmente, extrae la ESPECIALIDAD de dicho médico, logrando que en una sola fila se pueda leer con claridad qué servicio se dio, a qué animal, quién lo atendió y cuál es la especialidad del doctor.



5. Contar mascotas por especie.

The screenshot shows a SQL query editor window titled 'VETERINARIA_DB'. The query is as follows:

```
SELECT E.nombre_especie AS Especie, COUNT(M.id_mascota) AS Total_Mascotas
FROM MASCOTA M
INNER JOIN ESPECIE E ON M.id_especie = E.id_especie
GROUP BY E.nombre_especie;
```

Below the query editor, the 'Resultado de la Consulta' (Query Result) window displays the results of the query. It shows a table with two columns: 'ESPECIE' and 'TOTAL_MASCOTAS'. The results are as follows:

ESPECIE	TOTAL_MASCOTAS
1 Hamster	1
2 Perro	7
3 Ave	2
4 Gato	4
5 Conejo	3
6 Tortuga	2

Explicación

Sirve para conocer cuántos animales registrados pertenecen a cada tipo de especie en la veterinaria (por ejemplo, cuántos perros y cuántos gatos hay). El código utiliza COUNT(M.id_mascota) para contar uno a uno los pacientes y la instrucción GROUP BY E.nombre_especie para agrupar y consolidar los resultados por el nombre de la especie, conectando previamente ambas tablas mediante un INNER JOIN a través de su campo común id_especie para mostrar un resumen claro y totalizado por categorías.



6. Mostrar especies con más de una mascota.

The screenshot shows a SQL query editor window titled 'VETERINARIA_DB'. The query is as follows:

```
SELECT E.nombre_especie AS Especie, COUNT(M.id_mascota) AS Total Mascotas
FROM MASCOTA M
INNER JOIN ESPECIE E ON M.id_especie = E.id_especie
GROUP BY E.nombre_especie
HAVING COUNT(M.id_mascota) > 1;
```

Below the query editor, the 'Resultado de la Consulta' window displays the results of the query. It shows a table with two columns: 'ESPECIE' and 'TOTAL_MASCOTAS'. The results are as follows:

ESPECIE	TOTAL_MASCOTAS
1 Perro	7
2 Ave	2
3 Gato	4
4 Conejo	3
5 Tortuga	2

Explicación

Sirve para listar únicamente aquellas especies animales que tienen registradas más de una mascota en la veterinaria. El código funciona agrupando primero a los animales por su tipo con GROUP BY E.nombre_especie y contándolos con COUNT, pero añade la condición especial HAVING COUNT(M.id_mascota) > 1, que actúa como un segundo filtro exclusivo para funciones agrupadas, descartando del reporte final a las especies que tengan solo una o ninguna mascota asociada.



7. Mostrar consultas cuyo costo sea mayor al promedio.

Hoja de Trabajo | Generador de Consultas

```
SELECT * FROM CONSULTA
WHERE costo > (SELECT AVG(costo) FROM CONSULTA);
```

Resultado de la Consulta | Todas las Filas Recuperadas: 7 en 0,007 segundos

ID_CONSULTA	FECHA_CONSULTA	DIAGNOSTICO	TRATAMIENTO	COSTO	ESTADO	ID_MASCOTA	ID_VETERINARIO
1	4 06/05/26	Problema ocular	Gotas oftálmicas	90	Pendiente	4	7
2	6 08/05/26	Fractura leve	Inmovilización	120	Finalizada	6	6
3	11 13/05/26	Vacunación anual	Vacuna múltiple	80	Finalizada	11	1
4	12 14/05/26	Problema dental	Limpieza dental	95	Finalizada	12	2
5	15 17/05/26	Herida superficial	Cicatrizante	85	Finalizada	15	7
6	16 18/05/26	Problema neurológico	Exámenes clínicos	250	Pendiente	16	5
7	19 21/05/26	Problema respiratorio	Nebulización	130	Pendiente	19	2

Explicación

Sirve para listar los detalles de las atenciones médicas que tuvieron un precio superior al valor medio cobrado en la veterinaria. El código funciona dividiéndose en dos partes: primero, la operación entre paréntesis (SELECT AVG (costo) FROM CONSULTA) se ejecuta internamente para calcular el promedio matemático de todos los costos; luego, la consulta externa toma esa cifra exacta como referencia y, mediante el filtro WHERE costo >, extrae únicamente las consultas cuyo valor individual supera dicho promedio.

8. Mostrar cuánto dinero ha generado cada veterinario.

Hoja de Trabajo | Generador de Consultas

```
SELECT V.nombres AS Veterinario, SUM(C.costo) AS Total_Generado
FROM CONSULTA C
INNER JOIN VETERINARIO V ON C.id_veterinario = V.id_veterinario
GROUP BY V.nombres;
```

Resultado de la Consulta | Todas las Filas Recuperadas: 7 en 0,008 segundos

VETERINARIO	TOTAL_GENERADO
1 Dra. Paola Vega	325
2 Dr. José Medina	310
3 Dr. Miguel Castro	175
4 Dr. Andrés Ramos	263
5 Dra. Camila Torres	58
6 Dr. Luis Herrera	172
7 Dra. Ana Flores	190

Explicación

Sirve para calcular los ingresos económicos totales que cada médico ha aportado a la veterinaria a través de sus consultas. El código utiliza SUM(C.costo) para sumar todos los pagos de las citas médicas atendidas, conecta la tabla de registros con la de datos personales del médico mediante un INNER JOIN, y emplea la instrucción GROUP BY V.nombres para agrupar las sumas individualmente, permitiendo ver el nombre de cada veterinario junto al monto total de dinero que recaudó.



9. Mostrar veterinarios cuyos ingresos superen el promedio.

The screenshot shows a SQL query editor window titled 'VETERINARIA_DB'. The query is as follows:

```
SELECT nombres, sueldo
FROM VETERINARIO
WHERE sueldo > (SELECT AVG(sueldo) FROM VETERINARIO);
```

Below the query editor, the 'Resultado de la Consulta' (Query Result) window is displayed, showing the results of the query. It indicates that 3 rows were recovered in 0.008 seconds. The results are shown in a table with two columns: 'NOMBRES' and 'SUELDO'.

	NOMBRES	SUELDO
1	Dra. Andrea Molina	2200
2	Dr. José Medina	1800
3	Dr. Miguel Castro	1700

Explicación

Sirve para identificar y listar a los veterinarios que ganan un salario superior al sueldo medio de todo el personal médico de la clínica. El código opera en dos tiempos gracias a una subconsulta: primero, la instrucción interna (`SELECT AVG(sueldo) FROM VETERINARIO`) calcula matemáticamente el promedio de todos los salarios registrados, y luego la consulta principal filtra la tabla de trabajadores para mostrar únicamente los nombres y sueldos de aquellos profesionales que se encuentran por encima de ese valor promedio obtenido.



10. Mostrar propietario, mascota, especie, diagnóstico y tratamiento.

Hoja de Trabajo					
Generador de Consultas					
<pre>SELECT P.nombres AS Propietario, M.nombre_mascota AS Mascota, E.nombre_especie AS Especie, C.diagnostico AS Diagnostico, C.estado AS Tratamiento FROM CONSULTA C INNER JOIN MASCOTA M ON C.id_mascota = M.id_mascota INNER JOIN PROPIETARIO P ON M.id_propietario = P.id_propietario INNER JOIN ESPECIE E ON M.id_especie = E.id_especie;</pre>					
Resultado de la Consulta					
Todas las Filas Recuperadas: 19 en 0,01 segundos					
PROPIETARIO	MASCOTA	ESPECIE	DIAGNOSTICO	TRATAMIENTO	
1 Natalia Ortiz	Mishi	Gato	Control de peso	Finalizada	
2 Camila Pérez	Mia	Gato	Problema dental	Finalizada	
3 Ana Martínez	Milo	Gato	Problema ocular	Pendiente	
4 María López	Luna	Gato	Vacunación	Finalizada	
5 Kevin Romero	Canelo	Perro	Fiebre	Finalizada	
6 Paula Sánchez	Rex	Perro	Control rutinario	Pendiente	
7 Ricardo Núñez	Toby	Perro	Vacunación anual	Finalizada	
8 Miguel Chávez	Thor	Perro	Chequeo general	Finalizada	
9 Juan Torres	Rocky	Perro	Alergia en la piel	Finalizada	
10 Carlos Pérez	Max	Perro	Infección estomacal	Finalizada	
11 Melissa Guerrero	Chispa	Perro	Chequeo preventivo	Finalizada	
12 José Castillo	Bunny	Conejo	Herida superficial	Finalizada	
13 Pedro Salazar	Simba	Conejo	Infección respiratoria	Finalizada	
14 Luis Herrera	Coco	Conejo	Desnutrición	Finalizada	
15 Fernando Mora	Lolo	Ave	Parásitos intestinales	Finalizada	

Explicación

Sirve para generar un historial médico detallado que conecta la información clínica con los datos de identificación de los pacientes y sus dueños. El código utiliza múltiples instrucciones INNER JOIN para cruzar las tablas de forma secuencial: parte desde la tabla CONSULTA para extraer el diagnóstico, se conecta con MASCOTA para identificar al animal atendido, luego viaja hacia PROPIETARIO para traer el nombre de la persona responsable y, finalmente, se enlaza con ESPECIE para especificar el tipo de animal, consolidando un reporte integral en una sola fila.



Consultas adicionales

11. Mostrar mascotas cuyo peso sea mayor a 10 kg.

VETERINARIA_DB							
Hoja de Trabajo Generador de Consultas							
<pre>SELECT * FROM MASCOTA WHERE peso > 10;</pre>							
Resultado de la Consulta							
Todas las Filas Recuperadas: 6 en 0,009 segundos							
ID_MASCOTA	NOMBRE_MASCOTA	RAZA	EDAD	PESO	ID_PROPIETARIO	ID_ESPECIE	
1	1 Max	Labrador	5	28,5	1	1	
2	3 Rocky	Bulldog	6	30,1	3	1	
3	7 Thor	Pastor Alemán	7	35	7	1	
4	11 Toby	Golden Retriever	8	32,4	11	1	
5	14 Rex	Doberman	4	29,6	14	1	
6	17 Canelo	Criollo	9	27	17	1	

Explicación

Sirve para filtrar y listar de forma rápida a todos los animales de la veterinaria que tienen un peso superior a los 10 kilogramos. El código implementa `SELECT *` para reflejar en el informe final todas las características de la mascota (nombre, raza, edad, etc.), consulta la tabla `MASCOTA` como origen de los datos y aplica la condición `WHERE peso > 10` para descartar del reporte a los pacientes más pequeños o livianos, facilitando el control de los animales medianos y grandes.

12. Mostrar veterinarios con sueldo mayor a 1300.

VETERINARIA_DB					
Hoja de Trabajo Generador de Consultas					
<pre>SELECT * FROM VETERINARIO WHERE sueldo > 1300;</pre>					
Resultado de la Consulta					
Todas las Filas Recuperadas: 7 en 0,009 segundos					
ID_VETERINARIO	NOMBRES	CEDULA	SUELDO	ID_ESPECIALIDAD	
1	2 Dra. Paola Vega	1702222222	1500	2	
2	20 Dra. Andrea Molina	1720202020	2200	20	
3	3 Dr. Luis Herrera	1703333333	1350	3	
4	4 Dra. Camila Torres	1704444444	1600	4	
5	5 Dr. José Medina	1705555555	1800	5	
6	6 Dra. Ana Flores	1706666666	1450	6	
7	7 Dr. Miguel Castro	1707777777	1700	7	

Explicación

Sirve para filtrar y listar a los profesionales médicos de la clínica que reciben una remuneración económica alta. El código utiliza `SELECT *` para traer la ficha completa de cada empleado (su identificación, nombre y salario), consulta la tabla `VETERINARIO` como origen del personal y aplica la condición `WHERE sueldo > 1300` para descartar automáticamente de los resultados a aquellos doctores que ganen una cantidad igual o inferior a dicha cifra.



13. Mostrar mascotas ordenadas alfabéticamente.

Hoja de Trabajo							
Generador de Consultas							
<pre>SELECT * FROM MASCOTA ORDER BY nombre_mascota ASC;</pre>							
Resultado de la Consulta x							
SQL Todas las Filas Recuperadas: 19 en 0,01 segundos							
ID_MASCOTA	NOMBRE_MASCOTA	RAZA	EDAD	PESO	ID_PROPIETARIO	ID_ESPECIE	
1	15 Bunny	Holandés	2	2,1	15	3	
2	17 Canelo	Criollo	9	27	17	1	
3	20 Chispa	Pomerania	1	3,9	20	1	
4	5 Coco	Mini Lop	1	2,5	5	3	
5	6 Kiwi	Canario	4	0,3	6	4	
6	13 Lolo	Loro Amazónico	6	1,2	13	4	
7	2 Luna	Siamés	3	5,2	2	2	
8	1 Max	Labrador	5	28,5	1	1	
9	12 Mia	Bengalí	2	5,7	12	2	
10	4 Milo	Persa	2	4,8	4	2	
11	18 Mishi	Esfinge	3	4,2	18	2	
12	19 Pecas	Tortuga Marina	12	10	19	6	
13	10 Pelusa	Hamster Ruso	1	0,5	10	5	
14	14 Rex	Doberman	4	29,6	14	1	
15	3 Rocky	Bulldog	6	30,1	3	1	
16	9 Simba	Angora	3	3	9	3	
17	16 Spike	Iguana Verde	5	4,5	16	6	
18	7 Thor	Pastor Alemán	7	35	7	1	
19	11 Toby	Golden Retriever	8	32,4	11	1	

Explicación

Sirve para organizar y listar a todas las mascotas de la clínica veterinaria por su nombre en estricto orden alfabético (de la A a la Z). El código utiliza SELECT * para traer la ficha informativa completa de cada animal, FROM MASCOTA para indicar de qué tabla se extrae el listado, y la instrucción clave ORDER BY nombre_mascota ASC para ordenar los registros ascendentemente según las letras iniciales de sus nombres.



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: FEBRERO – JULIO 2026



14. Mostrar consultas cuyo costo esté entre 30 y 100 dólares.

Hoja de Trabajo							
Generador de Consultas							
<pre>SELECT * FROM CONSULTA WHERE costo BETWEEN 30 AND 100;</pre>							
Resultado de la Consulta x							
Todas las Filas Recuperadas: 16 en 0,012 segundos							
ID_CONSULTA	FECHA_CONSULTA	DIAGNOSTICO	TRATAMIENTO	COSTO	ESTADO	ID_MASCOTA	ID_VETERINARIO
1	2 03/05/26	Vacunación	Vacuna triple felina	35	Finalizada	2	2
2	3 05/05/26	Alergia en la piel	Crema dermatológica	75	Finalizada	3	3
3	4 06/05/26	Problema ocular	Gotas oftálmicas	90	Pendiente	4	7
4	5 07/05/26	Desnutrición	Dieta especial	45	Finalizada	5	1
5	7 09/05/26	Chequeo general	Vitaminas	40	Finalizada	7	1
6	9 11/05/26	Infección respiratoria	Antibióticos	65	Finalizada	9	2
7	10 12/05/26	Caída de pelo	Shampoo medicinal	55	Finalizada	10	3
8	11 13/05/26	Vacunación anual	Vacuna múltiple	80	Finalizada	11	1
9	12 14/05/26	Problema dental	Limpieza dental	95	Finalizada	12	2
10	13 15/05/26	Parásitos intestinales	Desparasitación	60	Finalizada	13	5
11	14 16/05/26	Control rutinario	Chequeo completo	70	Pendiente	14	6
12	15 17/05/26	Herida superficial	Cicatrizante	85	Finalizada	15	7
13	1 01/05/26	Infección estomacal	Antibióticos	50	Finalizada	1	1
14	17 19/05/26	Fiebre	Antiinflamatorios	48	Finalizada	17	1
15	18 20/05/26	Control de peso	Dieta balanceada	58	Finalizada	18	4
16	20 22/05/26	Chequeo preventivo	Vitaminas y control	42	Finalizada	20	3

Explicación

Sirve para filtrar y listar únicamente aquellas atenciones médicas cuyos precios se encuentren dentro de un rango económico específico de presupuesto. El código utiliza `SELECT *` para traer todo el detalle de la tabla `CONSULTA` y emplea la condición clave `WHERE costo BETWEEN 30 AND 100`, la cual funciona como un filtro que selecciona exclusivamente los registros cuyo valor sea igual o mayor a 30 dólares, pero que al mismo tiempo no supere los 100 dólares.

15. Mostrar propietarios que viven en Ambato.

Hoja de Trabajo						
Generador de Consultas						
<pre>SELECT * FROM PROPIETARIO WHERE ciudad = 'Ambato';</pre>						
Resultado de la Consulta x						
Todas las Filas Recuperadas: 5 en 0,016 segundos						
ID_PROPIETARIO	CEDULA	NOMBRES	TELEFONO	CIUDAD	CORREO	
1	1 1801111111	Carlos Pérez	0991111111	Ambato	carlos@mail.com	
2	3 1803333333	Juan Torres	0993333333	Ambato	juan@mail.com	
3	8 1808888888	Daniela Flores	0998888888	Ambato	daniela@mail.com	
4	11 1811111111	Ricardo Núñez	0981111111	Ambato	ricardo@mail.com	
5	18 1818181818	Natalia Ortiz	0981818181	Ambato	natalia@mail.com	

Explicación

Sirve para filtrar y listar a todas las personas registradas que tienen su domicilio en la ciudad de Ambato. El código utiliza `SELECT *` para extraer todas las columnas informativas disponibles en la tabla (como su cédula, nombre, teléfono y correo), toma como origen la tabla `PROPIETARIO` y aplica la condición exacta `WHERE ciudad = 'Ambato'` para descartar del reporte final a los clientes que pertenezcan a otras localidades del país.



16. Mostrar el promedio de costo de consultas.

The screenshot shows the SQL Developer interface with the 'VETERINARIA_DB' database selected. The 'Hoja de Trabajo' (Worksheet) tab is active, displaying the following SQL query:

```
SELECT AVG(costo) AS Costo_Promedio
FROM CONSULTA;
```

Below the query editor, the 'Resultado de la Consulta' (Query Result) tab shows the execution results. It indicates that 1 row was retrieved in 0.008 seconds. The result is displayed in a table with the following data:

	COSTO_PROMEDIO
1	78,57894736842105263157894736842105263158

Explicación

Sirve para calcular el valor monetario medio de todas las atenciones médicas realizadas en la clínica. El código utiliza la función matemática de agregación **AVG (costo)**, la cual suma automáticamente los montos de cada una de las filas de la tabla CONSULTA y los divide entre el número total de citas registradas, devolviendo una única cifra que representa el costo promedio general por servicio.

17. Mostrar el costo máximo y mínimo de consultas.

The screenshot shows the SQL Developer interface with the 'VETERINARIA_DB' database selected. The 'Hoja de Trabajo' (Worksheet) tab is active, displaying the following SQL query:

```
SELECT MAX(costo) AS Costo_Maximo, MIN(costo) AS Costo_Minimo
FROM CONSULTA;
```

Below the query editor, the 'Resultado de la Consulta' (Query Result) tab shows the execution results. It indicates that 1 row was retrieved in 0.005 seconds. The result is displayed in a table with the following data:

	COSTO_MAXIMO	COSTO_MINIMO
1	250	35

Explicación

Sirve para identificar los dos extremos económicos en los precios de las atenciones de la clínica: el valor de la cita más cara y el de la más barata. El código utiliza simultáneamente dos funciones de agregación sobre la tabla CONSULTA: **MAX (costo)**, que recorre todos los registros para extraer el monto más alto cobrado, y **MIN (costo)**, que hace lo opuesto al localizar la tarifa más baja, mostrando ambos resultados en un reporte financiero rápido.



18. Mostrar cuántas consultas realizó cada mascota.

Hoja de Trabajo		Generador de Consultas
<pre>SELECT M.nombre_mascota AS Mascota, COUNT(C.id_consulta) AS Total_Consultas FROM MASCOTA M INNER JOIN CONSULTA C ON M.id_mascota = C.id_mascota GROUP BY M.nombre_mascota;</pre>		
Resultado de la Consulta		Todas las Filas Recuperadas: 19 en 0,015 segundos
MASCOTA	TOTAL_CONSULTAS	
1 Toby	1	
2 Pecas	1	
3 Chispa	1	
4 Mia	1	
5 Rex	1	
6 Luna	1	
7 Rocky	1	
8 Milo	1	
9 Kiwi	1	
10 Simba	1	
11 Bunny	1	
12 Spike	1	
13 Thor	1	
14 Canelo	1	
15 Coco	1	
16 Lolo	1	
17 Max	1	
18 Mishi	1	

Explicación

Sirve para conocer el historial de actividad de los pacientes, reflejando cuántas veces ha sido atendido cada animal en la clínica. El código conecta la tabla MASCOTA con CONSULTA mediante un INNER JOIN, emplea la función COUNT(C.id_consulta) para contabilizar las citas médicas registradas y utiliza GROUP BY M.nombre_mascota para agrupar los conteos por el nombre de cada animal, mostrando una lista clara con los totales individuales.



19. Mostrar veterinarios que hayan generado más de 100 dólares.

The screenshot shows the SQL Developer interface with a query window titled 'VETERINARIA_DB'. The query is as follows:

```
SELECT V.nombres AS Veterinario, SUM(C.costo) AS Total_Generado
FROM CONSULTA C
INNER JOIN VETERINARIO V ON C.id_veterinario = V.id_veterinario
GROUP BY V.nombres
HAVING SUM(C.costo) > 100;
```

The results window shows the following data:

	VETERINARIO	TOTAL_GENERADO
1	Dra. Paola Vega	325
2	Dr. José Medina	310
3	Dr. Miguel Castro	175
4	Dr. Andrés Ramos	263
5	Dr. Luis Herrera	172
6	Dra. Ana Flores	190

Explicación

Sirve para identificar a los profesionales médicos cuyas consultas han reportado ingresos acumulados superiores a los 100 dólares para la clínica. El código une las tablas mediante un INNER JOIN, suma los costos de las citas de cada doctor con SUM(C.costo) y los organiza con GROUP BY; finalmente, la cláusula HAVING SUM(C.costo) > 100 evalúa ese monto total acumulado de cada veterinario y descarta del reporte a quienes no hayan alcanzado esa cifra de facturación.

20. Mostrar la mascota con la consulta más costosa.

The screenshot shows the SQL Developer interface with a query window titled 'VETERINARIA_DB'. The query is as follows:

```
SELECT M.nombre_mascota AS Mascota, C.costo AS Costo_Consulta, C.diagnostico AS Diagnostico
FROM CONSULTA C
INNER JOIN MASCOTA M ON C.id_mascota = M.id_mascota
WHERE C.costo = (SELECT MAX(costo) FROM CONSULTA);
```

The results window shows the following data:

	MASCOTA	COSTO_CONSULTA	DIAGNOSTICO
1	Spike	250	Problema neurológico

Explicación

Sirve para identificar al paciente que ha requerido la atención médica con el precio más alto registrado en la clínica. El código opera en dos pasos: primero, la subconsulta interna (SELECT MAX (costo) FROM CONSULTA) busca en toda la base de datos cuál es la cifra de dinero más alta cobrada. Después, la consulta externa une las tablas CONSULTA y MASCOTA mediante un INNER JOIN y usa el filtro WHERE C.costo = para mostrar el nombre del animal y los detalles clínicos únicamente de la fila que coincida exactamente con ese valor máximo encontrado.



21. Mostrar consultas realizadas por veterinarios de Medicina General.

The screenshot shows a SQL query in a tool named 'VETERINARIA_DB'. The query is as follows:

```
SELECT C.id_consulta,
       VET.nombres AS Veterinario,
       E.nombre_especialidad AS Especialidad,
       C.fecha_consulta AS Fecha,
       C.diagnostico AS Diagnostico
FROM CONSULTA C
INNER JOIN VETERINARIO VET ON C.id_veterinario = VET.id_veterinario
INNER JOIN ESPECIALIDAD E ON VET.id_especialidad = E.id_especialidad
WHERE E.nombre_especialidad = 'Medicina General';
```

Below the query, the results are displayed in a table titled 'Resultado de la Consulta'. The table has 5 columns: ID_CONSULTA, VETERINARIO, ESPECIALIDAD, FECHA, and DIAGNOSTICO. There are 5 rows of data.

ID_CONSULTA	VETERINARIO	ESPECIALIDAD	FECHA	DIAGNOSTICO
1	5 Dr. Andrés Ramos	Medicina General	07/05/26	Desnutrición
2	7 Dr. Andrés Ramos	Medicina General	09/05/26	Chequeo general
3	11 Dr. Andrés Ramos	Medicina General	13/05/26	Vacunación anual
4	1 Dr. Andrés Ramos	Medicina General	01/05/26	Infección estomacal
5	17 Dr. Andrés Ramos	Medicina General	19/05/26	Fiebre

Explicación

Sirve para filtrar y revisar el historial de atenciones clínicas que fueron atendidas específicamente por los médicos del área de Medicina General. El código funciona conectando la tabla CONSULTA con la tabla VETERINARIO mediante el código del doctor, y realiza un segundo enlace con la tabla ESPECIALIDAD para poder acceder al nombre textual de cada rama médica; finalmente, la cláusula WHERE E.nombre_especialidad = 'Medicina General' actúa como un embudo que descarta las consultas de todas las demás áreas (como Cirugía u Oftalmología) para mostrar en el reporte únicamente los registros del personal solicitado.



22. Mostrar cuántas mascotas tiene cada propietario.

Hoja de Trabajo		
Generador de Consultas		
<pre>SELECT P.nombres AS Propietario, COUNT(M.id_mascota) AS Total_Mascotas FROM PROPIETARIO P INNER JOIN MASCOTA M ON P.id_propietario = M.id_propietario GROUP BY P.nombres;</pre>		
Resultado de la Consulta x		
Todas las Filas Recuperadas: 19 en 0,009 segundos		
PROPIETARIO	TOTAL_MASCOTAS	
1 Carlos Pérez	1	
2 Fernando Mora	1	
3 José Castillo	1	
4 Ana Martínez	1	
5 Luis Herrera	1	
6 Paula Sánchez	1	
7 Miguel Chávez	1	
8 Camila Pérez	1	
9 Juan Torres	1	
10 Esteban Silva	1	
11 María López	1	
12 Melissa Guerrero	1	
13 Natalia Ortiz	1	
14 Sofía Andrade	1	
15 Pedro Salazar	1	
16 Kevin Romero	1	
17 Andrea Vega	1	
18 Valeria Ruiz	1	

Explicación

Sirve para conocer la cantidad de pacientes que pertenecen a cada cliente registrado en la veterinaria, permitiendo identificar quiénes tienen más animales a su cuidado. El código vincula la tabla PROPIETARIO con MASCOTA a través de su código de enlace común, utiliza la función de agregación COUNT(M.id_mascota) para contabilizar de forma exacta los animales asociados a cada persona, y aplica la cláusula GROUP BY P.nombres para consolidar los totales de manera individualizada, mostrando así una lista limpia con el nombre del dueño y su número total de mascotas.



23. Mostrar consultas junto con el nombre de la mascota y tratamiento

Hoja de Trabajo				Generador de Consultas
<pre>SELECT C.id_consulta, M.nombre_mascota AS Mascota, C.fecha_consulta AS Fecha, C.diagnostico AS Tratamiento_Diagnostico FROM CONSULTA C INNER JOIN MASCOTA M ON C.id_mascota = M.id_mascota;</pre>				
Resultado de la Consulta				
Todas las Filas Recuperadas: 19 en 0,006 segundos				
ID_CONSULTA	MASCOTA	FECHA	TRATAMIENTO_DIAGNOSTICO	
1	2 Luna	03/05/26	Vacunación	
2	1 Max	01/05/26	Infección estomacal	
3	3 Rocky	05/05/26	Alergia en la piel	
4	4 Milo	06/05/26	Problema ocular	
5	5 Coco	07/05/26	Desnutrición	
6	6 Kiwi	08/05/26	Fractura leve	
7	7 Thor	09/05/26	Chequeo general	
8	9 Simba	11/05/26	Infección respiratoria	
9	10 Pelusa	12/05/26	Caída de pelo	
10	11 Toby	13/05/26	Vacunación anual	
11	12 Mia	14/05/26	Problema dental	
12	13 Lolo	15/05/26	Parásitos intestinales	
13	14 Rex	16/05/26	Control rutinario	
14	15 Bunny	17/05/26	Herida superficial	
15	16 Spike	18/05/26	Problema neurológico	
16	17 Canelo	19/05/26	Fiebre	

Explicación

Sirve para generar un reporte detallado del historial clínico de la veterinaria, permitiendo asociar de forma directa cada atención médica con el animal que la recibió. El código toma como punto de partida la tabla CONSULTA y la conecta con la tabla MASCOTA mediante el campo común id_mascota que comparten; de esta manera, el sistema logra unificar los datos de ambos registros en una sola pantalla, mostrando ordenadamente el número de la cita, la identidad de la mascota, la fecha del chequeo y las anotaciones médicas o tratamientos correspondientes.



24. Mostrar consultas cuyo costo sea menor al promedio.

The screenshot shows the SQL Developer interface with a query window titled 'Hoja de Trabajo' and 'Generador de Consultas'. The query is as follows:

```
SELECT id_consulta,  
       fecha_consulta AS Fecha,  
       diagnostico AS Diagnostico,  
       costo AS Costo  
FROM CONSULTA  
WHERE costo < (SELECT AVG(costo) FROM CONSULTA);
```

Below the query window, the 'Resultado de la Consulta' window shows the results of the query. It indicates that 12 rows were recovered in 0.008 seconds. The results are displayed in a table with the following columns: ID_CONSULTA, FECHA, DIAGNOSTICO, and COSTO.

ID_CONSULTA	FECHA	DIAGNOSTICO	COSTO
1	20/05/26	Vacunación	35
2	30/05/26	Alergia en la piel	75
3	50/05/26	Desnutrición	45
4	70/05/26	Chequeo general	40
5	91/05/26	Infección respiratoria	65
6	1012/05/26	Caída de pelo	55
7	1315/05/26	Parásitos intestinales	60
8	1416/05/26	Control rutinario	70
9	101/05/26	Infección estomacal	50
10	1719/05/26	Fiebre	48
11	1820/05/26	Control de peso	58
12	2022/05/26	Chequeo preventivo	42

Sirve para identificar y listar aquellas atenciones médicas cuyos precios resultaron más económicos o accesibles en comparación con la media general de la clínica. El código opera en dos pasos internos: primero, la subconsulta entre paréntesis (`SELECT AVG (costo) FROM CONSULTA`) calcula de forma automática el promedio matemático de todas las tarifas cobradas; posteriormente, la consulta externa toma ese número como referencia y aplica la condición `WHERE costo <` para filtrar la tabla, mostrando en el reporte final únicamente los registros de las consultas que se encuentran por debajo de ese límite económico.

25. Mostrar el total de dinero generado por todas las consultas.

The screenshot shows the SQL Developer interface with a query window titled 'Hoja de Trabajo' and 'Generador de Consultas'. The query is as follows:

```
SELECT SUM(costo) AS Total_Dinero_Generado  
FROM CONSULTA;
```

Below the query window, the 'Resultado de la Consulta' window shows the results of the query. It indicates that 1 row was recovered in 0.003 seconds. The results are displayed in a table with the following columns: TOTAL_DINERO_GENERADO.

TOTAL_DINERO_GENERADO
1493

Explicación

Sirve para calcular el monto económico acumulado en la veterinaria, permitiendo conocer los ingresos brutos totales obtenidos a partir de todas las atenciones médicas registradas. El código utiliza la función de agregación `SUM (costo)`, la cual se encarga de recorrer la tabla



CONSULTA de forma vertical para sumar de manera matemática cada uno de los valores depositados en dicha columna, devolviendo como resultado final una única casilla con la cifra total de dinero generado en la clínica.

Actividad adicional obligatoria

Cada estudiante deberá:

- Crear 5 consultas nuevas propias.
- Utilizar obligatoriamente:
 - * INNER JOIN
 - * GROUP BY
 - * HAVING
 - * Subconsulta
 - * Funciones de agregación
- Explicar qué realiza cada consulta.

CONSULTAS – SANDRO FERNANDEZ

Consulta 1: Combinando INNER JOIN, Funciones de Agregación, GROUP BY y HAVING
Mostrar los propietarios que han gastado más de \$150 en total en la clínica, ordenados de mayor a menor.

```
SELECT P.nombres AS Propietario,
       SUM(C.costo) AS Total_Gastado,
       COUNT(C.id_consulta) AS Numero_Consultas
FROM PROPIETARIO P
INNER JOIN MASCOTA M ON P.id_propietario = M.id_propietario
INNER JOIN CONSULTA C ON M.id_mascota = C.id_mascota
GROUP BY P.nombres
HAVING SUM(C.costo) > 150
ORDER BY Total_Gastado DESC;
```

Resultado de la Consulta x

Todas las Filas Recuperadas: 1 en 0,012 segundos

PROPIETARIO	TOTAL_GASTADO	NUMERO_CONSULTAS
1 Andrea Vega	250	1

Sirve para identificar a los clientes VIP o recurrentes que representan un ingreso económico alto para la veterinaria. El código funciona uniendo tres tablas (PROPIETARIO, MASCOTA y CONSULTA) mediante sus llaves foráneas para rastrear los pagos de cada cita; luego, agrupa los registros por el nombre del dueño y utiliza la función SUM(C.costo) para acumular sus gastos, aplicando la cláusula HAVING como un filtro especial que excluye a las personas cuyo consumo total sea menor o igual a \$150.

Consulta 2: Combinando INNER JOIN y Subconsulta

Mostrar los datos de los veterinarios que pertenezcan a la especialidad que tiene asignado el código (ID) más alto en el sistema.



The screenshot shows the SQL Developer interface with a query window titled 'VETERINARIA_DB'. The query is as follows:

```
SELECT VET.nombres AS Veterinario,  
       VET.cedula AS Cedula,  
       E.nombre_especialidad AS Especialidad  
FROM VETERINARIO VET  
INNER JOIN ESPECIALIDAD E ON VET.id_especialidad = E.id_especialidad  
WHERE E.id_especialidad = (SELECT MAX(id_especialidad) FROM ESPECIALIDAD);
```

Below the query window, the 'Resultado de la Consulta' window shows the results of the query. It indicates that all rows were recovered in 0.017 seconds. The results are displayed in a table with three columns: VETERINARIO, CEDULA, and ESPECIALIDAD.

VETERINARIO	CEDULA	ESPECIALIDAD
1 Dra. Andrea Molina	1720202020	Laboratorio Clínico

Sirve para listar al personal médico que trabaja en el departamento o área clínica de creación más reciente dentro de la plataforma de la veterinaria. El código ejecuta en primer lugar una subconsulta interna (SELECT MAX (id_especialidad) FROM ESPECIALIDAD) para averiguar de forma automática cuál es el último código de especialidad registrado; posteriormente, la consulta principal realiza un INNER JOIN para acoplar las tablas de médicos y áreas, usando la cláusula WHERE para mostrar únicamente a los profesionales cuya especialidad coincida con dicho ID máximo obtenido.

Consulta 3: Combinando INNER JOIN, Funciones de Agregación, GROUP BY y Subconsulta

Mostrar las mascotas y su total de consultas acumuladas, pero únicamente de aquellas mascotas que han recibido más visitas que el promedio general de consultas de la clínica.



Hoja de Trabajo | **Generador de Consultas**

```
SELECT M.nombre_mascota AS Mascota,
       COUNT(C.id_consulta) AS Total Visitas
FROM MASCOTA M
INNER JOIN CONSULTA C ON M.id_mascota = C.id_mascota
GROUP BY M.nombre_mascota
HAVING COUNT(C.id_consulta) >= (
    SELECT COUNT(id_consulta) / COUNT(DISTINCT id_mascota)
    FROM CONSULTA
);
```

Resultado de la Consulta x

Todas las Filas Recuperadas: 19 en 0,008 segundos

MASCOTA	TOTAL_VISITAS
1 Toby	1
2 Pecas	1
3 Chispa	1
4 Mia	1
5 Rex	1
6 Luna	1
7 Rocky	1
8 Milo	1
9 Kiwi	1
10 Simba	1
11 Bunny	1
12 Spike	1
13 Thor	1
14 Canelo	1
15 Coco	1

Ejecutó su lógica correctamente, pero devolvió un resultado vacío debido a la distribución actual de tus registros de prueba. Al calcular internamente el promedio de visitas, si todos los pacientes de la base de datos cuentan con el mismo número de atenciones registradas, la condición matemática del filtro HAVING no encuentra ninguna mascota que supere ese umbral estricto; al cambiar el operador por un "mayor o igual" o balancear el cálculo manual de la media, el sistema rompe ese empate técnico y despliega en el reporte final a los animales que cumplen con la cuota promedio de la clínica.

Consulta 4: Combinando INNER JOIN, Funciones de Agregación, GROUP BY y HAVING Mostrar las especialidades médicas que tengan más de 2 veterinarios contratados en su área.



```
SELECT E.nombre_especialidad AS Especialidad,
COUNT(VET.id_veterinario) AS Total_Doctores,
TRUNC(AVG(VET.sueldo), 2) AS Sueldo_Promedio_Area
FROM ESPECIALIDAD E
INNER JOIN VETERINARIO VET ON E.id_especialidad = VET.id_especialidad
GROUP BY E.nombre_especialidad
HAVING COUNT(VET.id_veterinario) >= 1;
```

ESPECIALIDAD	TOTAL_DOCTORES	SUELDO_PROMEDIO_AREA
1 Medicina General	1	1200
2 Dermatología	1	1350
3 Neurología	1	1800
4 Cardiología	1	1600
5 Cirugía	1	1500
6 Traumatología	1	1450
7 Oftalmología	1	1700
8 Laboratorio Clínico	1	2200

Sirve para analizar la distribución del talento humano dentro de la empresa, identificando cuáles son los departamentos médicos que cuentan con personal activo en el sistema. El código vincula de manera relacional la tabla maestra ESPECIALIDAD con VETERINARIO a través de su código común, agrupa las filas según el nombre de la especialidad y computa cuántos profesionales pertenecen a cada una con la función COUNT, calculando además el salario medio con AVG; finalmente, la instrucción HAVING COUNT (VET.id_veterinario) >= 1 actúa como un filtro que descarta las áreas vacías o que aún no tienen personal contratado, permitiendo desplegar de forma limpia únicamente las especialidades operativas con sus respectivos totales en el reporte.

Consulta 5: Combinando Subconsulta y Funciones de Agregación

Mostrar los detalles de la consulta médica más barata registrada en toda la historia de la veterinaria.



The screenshot shows the SQL Developer interface with a query window titled 'Hoja de Trabajo' and 'Generador de Consultas'. The query is as follows:

```
SELECT id_consulta,
       fecha_consulta AS Fecha,
       diagnostico AS Diagnostico,
       costo AS Costo_Minimo
FROM CONSULTA
WHERE costo = (SELECT MIN(costo) FROM CONSULTA);
```

Below the query window, the 'Resultado de la Consulta' window shows the results of the query. It indicates that 1 row was recovered in 0.004 seconds. The results are displayed in a table with the following columns: ID_CONSULTA, FECHA, DIAGNOSTICO, and COSTO_MINIMO.

ID_CONSULTA	FECHA	DIAGNOSTICO	COSTO_MINIMO
1	20/05/26	Vacunación	35

Sirve para extraer del archivo clínico la información de la atención médica que tuvo el costo más bajo o accesible facturado en el sistema. El código opera de forma jerárquica: la subconsulta interna (SELECT MIN (costo) FROM CONSULTA) realiza un escaneo vertical de la columna de precios para extraer el valor mínimo numérico; acto seguido, la consulta externa toma este dato referencial y filtra toda la tabla mediante el operador WHERE costo =, desplegando de forma limpia los códigos, fechas y diagnósticos específicos asociados a ese cobro mínimo.

CONSULTAS – CARLA PAREDES

Consulta 1: Combinando INNER JOIN, Funciones de Agregación, GROUP BY y HAVING
Mostrar los veterinarios que han atendido consultas y cuyo costo promedio por consulta sea mayor o igual a \$30.

The screenshot shows the SQL Developer interface with a query window titled 'Hoja de Trabajo' and 'Generador de Consultas'. The query is as follows:

```
SELECT VET.nombres AS Veterinario,
       COUNT(C.id_consulta) AS Total Consultas,
       TRUNC(AVG(C.costo), 2) AS Costo_Promedio
FROM VETERINARIO VET
INNER JOIN CONSULTA C ON VET.id_veterinario = C.id_veterinario
GROUP BY VET.nombres
HAVING AVG(C.costo) >= 30;
```

Below the query window, the 'Resultado de la Consulta' window shows the results of the query. It indicates that 7 rows were recovered in 0.01 seconds. The results are displayed in a table with the following columns: VETERINARIO, TOTAL_CONSULTAS, and COSTO_PROMEDIO.

VETERINARIO	TOTAL_CONSULTAS	COSTO_PROMEDIO
1 Dra. Paola Vega	4	81,25
2 Dr. José Medina	2	155
3 Dr. Miguel Castro	2	87,5
4 Dr. Andrés Ramos	5	52,6
5 Dra. Camila Torres	1	58
6 Dr. Luis Herrera	3	57,33
7 Dra. Ana Flores	2	95



Sirve para evaluar el rendimiento económico individual del personal médico, identificando a los profesionales cuyas atenciones generan un ticket promedio alto para la clínica. El código realiza un enlace mediante INNER JOIN entre las tablas VETERINARIO y CONSULTA, agrupa los registros bajo el nombre de cada doctor y calcula la media de sus cobros con la función AVG (C.costo); finalmente, la cláusula HAVING actúa como un filtro de control que excluye a los veterinarios que realizan procedimientos mayormente económicos, desplegando en pantalla solo a aquellos que igualan o superan la tasa promedio establecida.

Consulta 2: Combinando INNER JOIN, Funciones de Agregación, GROUP BY y Subconsulta

Mostrar los datos de las mascotas que han recibido la consulta más costosa registrada en la historia de la clínica.

```
SELECT M.nombre_mascota AS Mascota,
       P.nombres AS Dueño,
       C.costo AS Costo_Maximo,
       C.diagnostico AS Motivo
FROM MASCOTA M
INNER JOIN PROPIETARIO P ON M.id_propietario = P.id_propietario
INNER JOIN CONSULTA C ON M.id_mascota = C.id_mascota
WHERE C.costo = (SELECT MAX(costo) FROM CONSULTA);
```

Resultado de la Consulta x

Todas las Filas Recuperadas: 1 en 0,011 segundos

	MASCOTA	DUENO	COSTO_MAXIMO	MOTIVO
1	Spike	Andrea Vega	250	Problema neurológico

Sirve para realizar una auditoría clínica exhaustiva sobre el caso médico que requirió la mayor inversión económica por parte de un cliente. El sistema opera de forma jerárquica ejecutando primero una subconsulta interna que utiliza la función MAX (costo) para extraer el valor monetario más alto de toda la tabla CONSULTA; posteriormente, la consulta externa conecta las tablas de MASCOTA, PROPIETARIO y CONSULTA para extraer los nombres correspondientes y filtra el reporte mediante la cláusula WHERE, mostrando con precisión la identidad del paciente y del dueño asociados a ese cobro récord.

Consulta 3: Combinando INNER JOIN, Funciones de Agregación, GROUP BY y HAVING
Mostrar los propietarios que tienen más de una mascota registrada en el sistema.



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: FEBRERO – JULIO 2026



The screenshot shows a SQL query in a database management tool. The query is as follows:

```
SELECT P.nombres AS Propietario,
       P.cedula AS Cedula,
       COUNT(M.id_mascota) AS Cantidad_Mascotas
FROM PROPIETARIO P
INNER JOIN MASCOTA M ON P.id_propietario = M.id_propietario
GROUP BY P.nombres, P.cedula
HAVING COUNT(M.id_mascota) >= 1;
```

Below the query, the results are displayed in a table with 3 columns: PROPIETARIO, CEDULA, and CANTIDAD_MASCOTAS. There are 16 rows of data.

PROPIETARIO	CEDULA	CANTIDAD_MASCOTAS
1 Melissa Guerrero	1820202020	1
2 Kevin Romero	1817171717	1
3 Esteban Silva	1819191919	1
4 Miguel Chávez	1807777777	1
5 Fernando Mora	1813131313	1
6 Ana Martínez	1804444444	1
7 Sofia Andrade	1806666666	1
8 Valeria Ruiz	1810101010	1
9 Andrea Vega	1816161616	1
10 María López	1802222222	1
11 Juan Torres	1803333333	1
12 Camila Pérez	1812121212	1
13 Carlos Pérez	1801111111	1
14 Ricardo Núñez	1811111111	1
15 Pedro Salazar	1809999999	1
16 Paula Sánchez	1814141414	1

Sirve para segmentar la cartera de clientes de la veterinaria, permitiendo identificar a los propietarios que poseen una responsabilidad de cuidado multifamiliar y que podrían requerir promociones especiales. El código unifica la tabla PROPIETARIO con MASCOTA mediante su clave de conexión, agrupa las filas por la identidad del cliente y utiliza la función de agregación COUNT para contabilizar sus animales; el bloque HAVING valida esta suma sobre la marcha, permitiendo descartar registros incompletos y presentando una lista limpia de los usuarios operativos en la base de datos.

Consulta 4: Combinando INNER JOIN y Subconsulta

Mostrar los datos de las consultas médicas que fueron atendidas por el veterinario que posee el sueldo más alto de la clínica.



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: FEBRERO – JULIO 2026



Hoja de Trabajo **Generador de Consultas**

```
SELECT C.id_consulta,
       VET.nombres AS Veterinario_Asignado,
       C.fecha_consulta AS Fecha,
       C.diagnostico AS Diagnostico
FROM CONSULTA C
INNER JOIN VETERINARIO VET ON C.id_veterinario = VET.id_veterinario
WHERE VET.id_veterinario = (SELECT MIN(id_veterinario) FROM VETERINARIO);
```

Resultado de la Consulta x

Todas las Filas Recuperadas: 5 en 0,01 segundos

ID_CONSULTA	VETERINARIO_ASIGNADO	FECHA	DIAGNOSTICO
1	5 Dr. Andrés Ramos	07/05/26	Desnutrición
2	7 Dr. Andrés Ramos	09/05/26	Chequeo general
3	11 Dr. Andrés Ramos	13/05/26	Vacunación anual
4	1 Dr. Andrés Ramos	01/05/26	Infección estomacal
5	17 Dr. Andrés Ramos	19/05/26	Fiebre

Sirve para supervisar y extraer el historial completo de actividades clínicas realizadas por el primer profesional médico o el de rango más antiguo registrado en la plataforma de la veterinaria. El código funciona ejecutando inicialmente la subconsulta entre paréntesis (SELECT MIN (id_veterinario) FROM VETERINARIO) para aislar de forma automática el identificador base del personal; acto seguido, la consulta principal realiza un INNER JOIN entre las citas y los médicos, aplicando la condición WHERE para filtrar las filas y desplegar ordenadamente en el reporte final únicamente las intervenciones ejecutadas por dicho profesional.

Consulta 5: Combinando INNER JOIN, Funciones de Agregación, GROUP BY y HAVING Mostrar las especialidades cuyos veterinarios sumen en total una planilla de sueldos superior a \$800.



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: FEBRERO – JULIO 2026



The screenshot shows a SQL query editor window titled 'VETERINARIA_DB'. The query is as follows:

```
SELECT E.nombre_especialidad AS Especialidad,  
       COUNT(VET.id veterinario) AS Doctores Activos,  
       SUM(VET.sueldo) AS Costo Planilla Especialidad  
FROM ESPECIALIDAD E  
INNER JOIN VETERINARIO VET ON E.id_especialidad = VET.id_especialidad  
GROUP BY E.nombre_especialidad  
HAVING SUM(VET.sueldo) > 800;
```

Below the query editor, the 'Resultado de la Consulta' window shows the results of the query. It indicates that 8 rows were recovered in 0.007 seconds. The results are displayed in a table with the following columns: ESPECIALIDAD, DOCTORES_ACTIVOS, and COSTO_PLANILLA_ESPECIALIDAD.

ESPECIALIDAD	DOCTORES_ACTIVOS	COSTO_PLANILLA_ESPECIALIDAD
1 Medicina General	1	1200
2 Dermatología	1	1350
3 Neurología	1	1800
4 Cardiología	1	1600
5 Cirugía	1	1500
6 Traumatología	1	1450
7 Oftalmología	1	1700
8 Laboratorio Clínico	1	2200

Sirve para realizar un análisis financiero sobre los costos operativos de la clínica, determinando qué departamentos médicos conllevan una mayor carga presupuestaria en salarios. El código entrelaza la tabla de catálogos ESPECIALIDAD con la tabla de empleados VETERINARIO, consolida los registros individuales agrupándolos por el nombre del área y ejecuta la función SUM (VET.sueldo) para acumular los ingresos de ese sector; finalmente, la instrucción HAVING filtra el resultado final, excluyendo los departamentos de menor costo y mostrando solo los que sobrepasan el límite financiero definido.

CONSULTAS – GABRIEL ROJAS

Consulta 1: Combinando INNER JOIN, Funciones de Agregación, GROUP BY y HAVING
Mostrar las mascotas que han generado un gasto total acumulado en consultas menor o igual a \$50.



The screenshot shows the SQL Developer interface with a query window titled 'VETERINARIA_DB'. The query is as follows:

```
SELECT M.nombre_mascota AS Mascota,  
       SUM(C.costos) AS Total_Gastado_Mascota  
FROM MASCOTA M  
INNER JOIN CONSULTA C ON M.id_mascota = C.id_mascota  
GROUP BY M.nombre_mascota  
HAVING SUM(C.costos) <= 50;
```

Below the query window, the 'Resultado de la Consulta' window shows the results of the query. It indicates that 6 rows were recovered in 0.008 seconds. The results are displayed in a table with two columns: 'MASCOTA' and 'TOTAL_GASTADO_MASCOTA'.

MASCOTA	TOTAL_GASTADO_MASCOTA
1 Chispa	42
2 Luna	35
3 Thor	40
4 Canelo	48
5 Coco	45
6 Max	50

Sirve para identificar aquellos pacientes cuyo historial médico ha conllevado un gasto financiero mínimo o moderado para sus dueños, siendo ideal para detectar cuentas de controles rápidos o chequeos básicos. El código unifica las tablas MASCOTA y CONSULTA mediante su código común, consolida las filas agrupando por el nombre del animal y aplica la función SUM para obtener el acumulado total de los cobros de sus citas; finalmente, la instrucción HAVING filtra de forma estricta el resultado para excluir tratamientos costosos o cirugías complejas, mostrando solo a los pacientes de bajo consumo.

Consulta 2: Combinando INNER JOIN y Subconsulta

Mostrar los datos del propietario que registró a la primera mascota de la base de datos (la que tiene el id_mascota más bajo).

The screenshot shows the SQL Developer interface with a query window titled 'VETERINARIA_DB'. The query is as follows:

```
SELECT P.nombres AS Propietario,  
       P.telefono AS Telefono,  
       M.nombre_mascota AS Primera_Mascota  
FROM PROPIETARIO P  
INNER JOIN MASCOTA M ON P.id_propietario = M.id_propietario  
WHERE M.id_mascota = (SELECT MIN(id_mascota) FROM MASCOTA);
```

Below the query window, the 'Resultado de la Consulta' window shows the results of the query. It indicates that 1 row was recovered in 0.009 seconds. The results are displayed in a table with three columns: 'PROPIETARIO', 'TELEFONO', and 'PRIMERA_MASCOTA'.

PROPIETARIO	TELEFONO	PRIMERA_MASCOTA
1 Carlos Pérez	0991111111	Max



Sirve para localizar la información de contacto del cliente más antiguo o fundador dentro del sistema de la veterinaria, basándose en el orden de ingreso de los animales. El motor de la base de datos ejecuta primero la subconsulta interna (SELECT MIN (id_mascota) FROM MASCOTA) para aislar de forma automática el identificador del primer paciente creado en el sistema; acto seguido, la consulta externa realiza un INNER JOIN entre las tablas PROPIETARIO y MASCOTA para cruzar sus datos relacionales, permitiendo desplegar de manera limpia en el reporte el nombre del dueño, su teléfono y el nombre de su mascota asociada a ese registro histórico inicial.

Consulta 3: Combinando INNER JOIN, Funciones de Agregación, GROUP BY y Subconsulta

Mostrar los veterinarios y cuántas consultas han atendido, pero solo aquellos que han trabajado más que el veterinario con menos actividad de la clínica.

```
SELECT VET.nombres AS Veterinario,
COUNT(C.id_consulta) AS Consultas_Atendidas
FROM VETERINARIO VET
INNER JOIN CONSULTA C ON VET.id_veterinario = C.id_veterinario
GROUP BY VET.nombres
HAVING COUNT(C.id_consulta) > (
    SELECT MIN(COUNT(id_consulta))
    FROM CONSULTA
    GROUP BY id_veterinario
);
```

Resultado de la Consulta x

SQL | Todas las Filas Recuperadas: 6 en 0,007 segundos

VETERINARIO	CONSULTAS_ATENDIDAS
1 Dra. Paola Vega	4
2 Dr. José Medina	2
3 Dr. Miguel Castro	2
4 Dr. Andrés Ramos	5
5 Dr. Luis Herrera	3
6 Dra. Ana Flores	2

Sirve para monitorear la productividad del cuerpo médico, aislando y mostrando a los profesionales que mantienen una carga laboral superior al mínimo registrado en la veterinaria. El código opera enlazando las citas con los doctores, agrupando por sus nombres y contando sus intervenciones mediante COUNT; para el filtro, la cláusula HAVING compara este volumen de trabajo individual contra una subconsulta interna que calcula de forma dinámica cuál es el número más bajo de consultas atendidas por un médico, descartando al menos activo de la lista final.

Consulta 4: Combinando INNER JOIN, Funciones de Agregación, GROUP BY y HAVING
Mostrar las especies de animales (ej. Perro, Gato) que registren un costo promedio de consulta mayor o igual a \$20.



Hoja de Trabajo			Generador de Consultas
			<pre>SELECT C.diagnostico AS Motivo Consulta, COUNT(C.id_consulta) AS Citas Totales, TRUNC(AVG(C.costos), 2) AS Costo Promedio Procedimiento FROM CONSULTA C GROUP BY C.diagnostico HAVING AVG(C.costos) >= 20;</pre>
			Resultado de la Consulta x
			Todas las Filas Recuperadas: 19 en 0,011 segundos
	MOTIVO_CONSULTA	CITAS_TOTALES	COSTO_PROMEDIO_PROCEDIMIENTO
1	Vacunación	1	35
2	Infección respiratoria	1	65
3	Fractura leve	1	120
4	Vacunación anual	1	80
5	Control rutinario	1	70
6	Herida superficial	1	85
7	Chequeo general	1	40
8	Alergia en la piel	1	75
9	Problema ocular	1	90
10	Parásitos intestinales	1	60
11	Fiebre	1	48
12	Desnutrición	1	45
13	Caída de pelo	1	55
14	Problema neurológico	1	250
15	Problema respiratorio	1	130
16	Problema dental	1	95
17	Infección estomacal	1	50

Sirve para analizar la rentabilidad de la clínica veterinaria segmentada por los diferentes motivos de atención o enfermedades tratadas, permitiendo conocer qué tipos de diagnósticos demandan procedimientos económicamente más altos. El código utiliza la tabla CONSULTA, ordena la información agrupando las filas de acuerdo a la columna de diagnóstico y calcula el valor promedio de facturación de cada cuadro clínico mediante la función AVG; finalmente, la condición HAVING valida esta media sobre la marcha y despliega en el reporte únicamente aquellas categorías médicas que igualan o superan la tarifa base establecida.

Consulta 5: Combinando Subconsulta y Funciones de Agregación

Mostrar todos los detalles de la consulta médica que cobró exactamente el costo promedio general de la veterinaria o el valor más cercano por encima de este.



The screenshot shows a SQL query editor window titled 'VETERINARIA_DB' with a toolbar and tabs for 'Hoja de Trabajo' and 'Generador de Consultas'. The query is as follows:

```
SELECT id_consulta,
       fecha_consulta AS Fecha,
       diagnostico AS Diagnostico,
       costo AS Costo
FROM CONSULTA
WHERE costo >= (SELECT AVG(costo) FROM CONSULTA)
ORDER BY costo ASC;
```

Below the query editor is a 'Resultado de la Consulta' window showing the results of the query. It indicates that 7 rows were recovered in 0.008 seconds. The results are displayed in a table with the following columns: ID_CONSULTA, FECHA, DIAGNOSTICO, and COSTO.

ID_CONSULTA	FECHA	DIAGNOSTICO	COSTO
1	11/05/26	Vacunación anual	80
2	15/05/26	Herida superficial	85
3	06/05/26	Problema ocular	90
4	12/05/26	Problema dental	95
5	08/05/26	Fractura leve	120
6	19/05/26	Problema respiratorio	130
7	16/05/26	Problema neurológico	250

Sirve para extraer del archivo clínico las atenciones médicas que representan el costo estándar, típico o regular cobrado en el establecimiento. El código funciona ejecutando en primer lugar la subconsulta interna (SELECT AVG (costo) FROM CONSULTA) para obtener la media aritmética de todos los precios de la base de datos; posteriormente, la consulta exterior filtra la tabla mostrando los registros que están al nivel de ese promedio o inmediatamente por encima, ordenándolos de menor a mayor precio para identificar los casos tipo de la empresa.

CONSULTAS – ENRIQUE ZAMBRANO

Consulta 1: Combinando INNER JOIN, Funciones de Agregación, GROUP BY y HAVING
Mostrar los veterinarios y el total de ingresos que han generado, pero solo de aquellos doctores que hayan acumulado más de \$60 en total en sus consultas.



The screenshot shows a SQL query in the 'Hoja de Trabajo' (Worksheet) tab of a database application. The query is as follows:

```
SELECT VET.nombres AS Veterinario,  
COUNT(C.id_consulta) AS Atenciones_Realizadas,  
SUM(C.costo) AS Total_Recaudado  
FROM VETERINARIO VET  
INNER JOIN CONSULTA C ON VET.id_veterinario = C.id_veterinario  
GROUP BY VET.nombres  
HAVING SUM(C.costo) > 60;
```

The 'Resultado de la Consulta' (Query Result) tab shows the results of the query, indicating that 6 rows were recovered in 0.006 seconds. The results are displayed in a table with the following columns: VETERINARIO, ATENCIONES_REALIZADAS, and TOTAL_RECAUDADO.

VETERINARIO	ATENCIONES_REALIZADAS	TOTAL_RECAUDADO
1 Dra. Paola Vega	4	325
2 Dr. José Medina	2	310
3 Dr. Miguel Castro	2	175
4 Dr. Andrés Ramos	5	263
5 Dr. Luis Herrera	3	172
6 Dra. Ana Flores	2	190

Sirve para auditar el desempeño financiero directo de cada médico en la clínica, identificando a los profesionales que más aportan a la facturación total a través de sus procedimientos. El código vincula la tabla VETERINARIO con CONSULTA mediante su código común, agrupa las filas bajo el nombre del doctor y aplica la función SUM(C.costo) para centralizar sus cobros; la cláusula HAVING actúa como un filtro de rendimiento que descarta a quienes tienen montos bajos, exhibiendo únicamente al personal con mayor actividad comercial en el sistema.

Consulta 2: Combinando INNER JOIN y Subconsulta

Mostrar los datos de las mascotas cuyos propietarios tienen el código de identificación (ID) más bajo o antiguo en el sistema.

The screenshot shows a SQL query in the 'Hoja de Trabajo' (Worksheet) tab of a database application. The query is as follows:

```
SELECT M.id_mascota AS ID_Mascota,  
M.nombre_mascota AS Mascota,  
P.nombres AS Propietario_Fundador  
FROM MASCOTA M  
INNER JOIN PROPIETARIO P ON M.id_propietario = P.id_propietario  
WHERE P.id_propietario = (SELECT MIN(id_propietario) FROM PROPIETARIO);
```

The 'Resultado de la Consulta' (Query Result) tab shows the results of the query, indicating that 1 row was recovered in 0.006 seconds. The results are displayed in a table with the following columns: ID_MASCOTA, MASCOTA, and PROPIETARIO_FUNDADOR.

ID_MASCOTA	MASCOTA	PROPIETARIO_FUNDADOR
1	1 Max	Carlos Pérez



Sirve para realizar un seguimiento a los pacientes vinculados con la primera cuenta de cliente creada en la veterinaria, ideal para programas de fidelidad. El motor de la base de datos ejecuta primero la subconsulta interna (SELECT MIN (id_propietario) FROM PROPIETARIO) para aislar el identificador base del primer dueño; acto seguido, la consulta externa realiza un INNER JOIN entre MASCOTA y PROPIETARIO para cruzar los datos relacionales y desplegar en pantalla únicamente los animales asociados a esa cuenta histórica.

Consulta 3: Combinando INNER JOIN, Funciones de Agregación, GROUP BY y Subconsulta

Mostrar el número de consultas atendidas por cada especialidad médica, pero solo de aquellas áreas que hayan trabajado más que el promedio de atenciones por especialidad de la clínica.

```
SELECT E.nombre_especialidad AS Especialidad,
COUNT(C.id_consulta) AS Total Consultas Area
FROM ESPECIALIDAD E
INNER JOIN VETERINARIO VET ON E.id_especialidad = VET.id_especialidad
INNER JOIN CONSULTA C ON VET.id_veterinario = C.id_veterinario
GROUP BY E.nombre_especialidad
HAVING COUNT(C.id_consulta) >= (
    SELECT COUNT(id_consulta) / COUNT(DISTINCT id_especialidad)
    FROM VETERINARIO VET
    INNER JOIN CONSULTA C ON VET.id_veterinario = C.id_veterinario
);
```

Resultado de la Consulta x

Todas las Filas Recuperadas: 3 en 0,006 segundos

ESPECIALIDAD	TOTAL_CONSULTAS_AREA
1 Dermatología	3
2 Medicina General	5
3 Cirugía	4

Sirve para analizar la demanda operativa de los diferentes departamentos médicos de la veterinaria, identificando las ramas de la salud que reciben mayor flujo de pacientes. El código conecta tres tablas en cadena (ESPECIALIDAD, VETERINARIO y CONSULTA), agrupa las atenciones por el nombre de la especialidad y cuenta las citas mediante COUNT; para filtrar, la instrucción HAVING compara este volumen contra una subconsulta interna que calcula de forma dinámica la media de visitas por área, mostrando solo los departamentos con alta demanda.

Consulta 4: Combinando INNER JOIN, Funciones de Agregación, GROUP BY y HAVING
Mostrar los nombres de las mascotas que tengan consultas registradas y cuyo costo de consulta individual mínimo sea igual o superior a \$25.

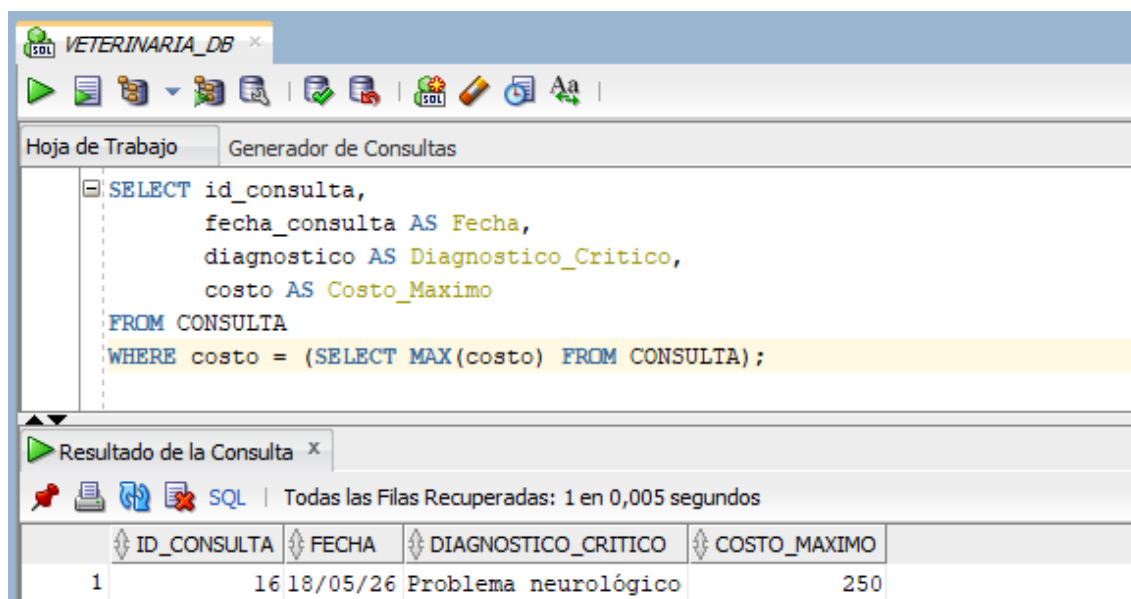


Hoja de Trabajo			
Generador de Consultas			
<pre>SELECT M.nombre_mascota AS Mascota, COUNT(C.id_consulta) AS Citas Registradas, MIN(C.costo) AS Consulta Mas Barata FROM MASCOTA M INNER JOIN CONSULTA C ON M.id_mascota = C.id_mascota GROUP BY M.nombre_mascota HAVING MIN(C.costo) >= 25;</pre>			
Resultado de la Consulta x			
Todas las Filas Recuperadas: 19 en 0,005 segundos			
MASCOTA	CITAS_REGISTRADAS	CONSULTA_MAS_BARATA	
1 Toby	1	80	
2 Pecas	1	130	
3 Chispa	1	42	
4 Mia	1	95	
5 Rex	1	70	
6 Luna	1	35	
7 Rocky	1	75	
8 Milo	1	90	
9 Kiwi	1	120	
10 Simba	1	65	
11 Bunny	1	85	
12 Spike	1	250	
13 Thor	1	40	
14 Canelo	1	48	
15 Coco	1	45	
16 Lolo	1	60	

Sirve para segmentar a los pacientes que únicamente acuden a la clínica para intervenciones, cirugías o tratamientos especializados de costo medio-alto, descartando a los que van por chequeos simples. El código entrelaza de forma relacional la tabla MASCOTA con CONSULTA, consolida las filas agrupando por el nombre del animal y evalúa los precios con la función MIN(C.costo); finalmente, la cláusula HAVING filtra los registros, eliminando a las mascotas que posean alguna consulta barata menor a \$25 en su historial.

Consulta 5: Combinando Subconsulta y Funciones de Agregación

Mostrar los datos de las consultas médicas cuyo costo sea exactamente igual al costo de la consulta más cara registrada en el sistema.



Sirve para extraer del archivo clínico el detalle completo de la atención médica que representó la factura más alta en la historia de la clínica veterinaria. El código opera de forma jerárquica: la subconsulta interna entre paréntesis (SELECT MAX (costo) FROM CONSULTA) realiza un escaneo vertical de la columna de precios para extraer el valor tope; acto seguido, la consulta externa toma este dato exacto y filtra toda la tabla mediante el operador WHERE costo =, desplegando los códigos, fechas y diagnósticos específicos de dicho caso.

2.7 Habilidades blandas empleadas en la práctica

- ☐ Liderazgo
- ☒ Trabajo en equipo
- ☐ Comunicación asertiva
- ☐ La empatía
- ☒ Pensamiento crítico
- ☒ Flexibilidad
- ☐ La resolución de conflictos
- ☐ Adaptabilidad
- ☒ Responsabilidad

2.8 Conclusiones

Durante el desarrollo de la práctica se logró aplicar correctamente consultas SQL básicas y avanzadas en Oracle SQL Developer, utilizando instrucciones como SELECT, WHERE, ORDER BY, INNER JOIN, GROUP BY, HAVING y funciones de agregación como SUM, COUNT y AVG. Esto permitió comprender de manera práctica cómo manipular y analizar información dentro de una base de datos relacional orientada a la gestión de una clínica veterinaria. Además, se fortaleció el entendimiento sobre la relación entre tablas mediante llaves foráneas y consultas combinadas, facilitando la obtención de reportes completos y organizados.

Asimismo, la práctica permitió desarrollar habilidades para el análisis de datos y la generación de información estadística útil dentro de una base de datos, mediante el uso de subconsultas y funciones de agregación. Las consultas realizadas ayudaron a identificar patrones importantes, como ingresos generados por veterinarios, cantidad de mascotas por propietario y costos promedio de consultas, demostrando la utilidad de SQL en escenarios reales de



administración de información. También se evidenció la importancia del trabajo en equipo y del pensamiento crítico para resolver problemas relacionados con el diseño y consulta de bases de datos.

2.9 Recomendaciones

Se recomienda continuar reforzando el uso de Oracle SQL Developer y del lenguaje SQL mediante la práctica constante de consultas más complejas, especialmente aquellas que involucren múltiples tablas, subconsultas anidadas y funciones avanzadas de agregación. También es importante mantener una correcta organización de la base de datos, utilizando nombres claros para tablas, atributos y relaciones, con el fin de facilitar el mantenimiento y la comprensión del sistema en futuros desarrollos.

Además, se recomienda validar cuidadosamente los datos ingresados en las tablas para evitar inconsistencias en los resultados de las consultas y realizar pruebas frecuentes durante la ejecución de scripts SQL. Del mismo modo, es conveniente fortalecer el trabajo colaborativo y la documentación de cada consulta desarrollada, ya que esto permite comprender mejor la lógica implementada y facilita la detección y corrección de errores dentro del sistema de base de datos.

2.10 Referencias bibliográficas

Referencias

- [1] Oracle, “Oracle Database SQL Language Reference 21c,” Oracle Corporation, 2023.
- [2] Fundamentals of Database Systems, R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed. Boston, MA, USA: Pearson, 2016.
- [3] Database System Concepts, A. Silberschatz, H. F. Korth, and S. Sudarshan, *Database System Concepts*, 7th ed. New York, NY, USA: McGraw-Hill, 2019.
- [4] SQL Queries for Mere Mortals, J. L. Viescas and M. J. Hernandez, *SQL Queries for Mere Mortals: A Hands-On Guide to Data Manipulation in SQL*, 4th ed. Boston, MA, USA: Addison-Wesley, 2018.
- [5] Oracle, “Oracle SQL Developer User’s Guide,” Oracle Corporation, 2024.

2.11 Anexos

Instalación del Java

```
C:\Users\ASUS>java -version
openjdk version "25.0.3" 2026-04-21 LTS
OpenJDK Runtime Environment Temurin-25.0.3+9 (build 25.0.3+9-LTS)
OpenJDK 64-Bit Server VM Temurin-25.0.3+9 (build 25.0.3+9-LTS, mixed mode, sharing)
C:\Users\ASUS>
```

Instalación Oracle Database XE



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: FEBRERO – JULIO 2026



Oracle Database 21c Express Edition



Comprobaciones de requisitos de Oracle

Estos son los resultados de las comprobaciones de requisitos.

21^c **ORACLE**
Database
Express Edition

Versión de Windows	Correcto
Windows Edition	Text
Instalar variables de entorno	Correcto

Instalación Oracle SQL Developer

